

Federated Learning-Based Task Offloading in a UAV-Aided Cloud Computing Mobile Network

Nipun Agarwal  and Sandeep Joshi , Senior Member, IEEE

Abstract—Unmanned aerial vehicles (UAVs), due to their flexibility in deployment, offer various advantages in the next-generation wireless networks. In this work, we study a UAV-assisted mobile edge computing network where a UAV is equipped with computing resources. The user equipment (UE) is capable of transferring a part of the computational workload to the UAV. We aim to reduce the maximum processing delay while adhering to the energy consumption limitation. This is accomplished by optimizing user scheduling, task offloading ratio, UAV's flight angle, and flight speed. We propose a computation offloading approach using federated learning based on decentralized federated averaging, taking into account the fact that this optimization problem is not convex, the state space is high-dimensional, and the action space is continuous. We determine the best strategy for offloading computing in an environment whose dynamics are challenging to regulate. We perform extensive simulation testing, and our findings indicate that the proposed method converges faster to the optimal solution. Furthermore, the proposed algorithm significantly improves the processing delay by about 20% at smaller task sizes and even higher for larger task sizes as compared to the baseline algorithms.

Index Terms—Computation offloading, edge computing, federated learning, machine learning, unmanned aerial vehicle.

I. INTRODUCTION

The user equipment (UE) in wireless communication networks have limited computational power and battery capacities of varying sizes. Mobile cloud computing (MCC) [1] is a relatively new concept that improves the computing and storage capacities of the UE and lowers energy consumption by offloading the computations to the cloud through mobile networks. However, as the cloud servers are physically located in remote locations from UE, the MCC can result in high transmission delays that negatively impact user experiences [1]. Mobile computing has recently transitioned to a new computing paradigm, called mobile edge computing (MEC) [2], to eliminate the latency caused by backhaul networks. MEC can move resources from cloud computing [2] and services located in the more immediate vicinity of the UE to efficiently reduce both the time in transmission and the amount of energy used. Further, the unmanned aerial vehicle (UAV)-assisted MEC system makes it possible for the devices located nearby to offload computing services from the UE to a UAV equipped with computing capabilities [2].

The first research on MEC primarily focused on improving the performance of a multi-element computing system in which computing

services are handled by a base station located at a fixed location [3]. In a multi-user, multi-server MEC system, a convex optimization approach for optimizing resource allocation and a low-complexity heuristic algorithm for task offloading are presented as potential solutions in [3]. Using a collaborative computation offloading and resource allocation optimization strategy in a cloud-assisted MEC system, the research in [3] endeavors to determine how best to maximize system usefulness, minimizing the computational burden of multiple-input-multiple-output-based multi-user MEC systems. It is impossible for the MEC services provided by permanent infrastructure to function correctly when there are a limited number of communication facilities or when unexpected natural catastrophes occur. In UAV-assisted MEC systems, the performance is severely affected by the limitation on UE computation capability and the blockages due to obstacles such as trees or buildings [4]. Adaptive link selection and task offloading concerns are of the utmost significance in the UAV-assisted MEC system. The works in [5], [6], [7] pay specific attention to UAV-assisted computing due to its high mobility and flexible deployment and study how the UAV-aided MEC systems allocate resources and design routes. A method based on the penalty dual decomposition optimization framework was developed in [5] to concurrently optimize UAV trajectory, compute offloading and user scheduling with the goal of decreasing processing latency among UEs in the UAV-assisted MEC system. The study did not consider the volatility of the connection established by the UAV and considered the only territory where the UAV was stationary after an initial flight. In [6], [7], the authors developed an approach to optimize the data allocation and UAV flight path together, conserving power, but they did not capture the time delay while data transmission, which led to data packages being dropped at the UE, and eventually leading to the loss of overall information.

Deep reinforcement learning (DRL)-based approaches are effective in determining the best policy in a dynamic edge system. Therefore, we adopt deep deterministic policy gradient (DDPG) [8] as a baseline control model for each UE device to reduce latency. The DRL-based approach needs a large amount of computing resources hence the deployment of DRL agents should be carefully considered. Further, there are various disadvantages to training the DRL agent. Firstly, the transmitted training data may contain sensitive information, particularly in applications like industrial informatics, which poses a privacy risk. Secondly, even if the training data is modified to safeguard privacy, the data received by the UAV is less pertinent and loses its applicability to a particular UE. Thirdly, transmitting large quantities of training data from UE to the UAV will cause the wireless channel to become overloaded. Furthermore, there are two deficiencies in instructing the DRL agent on the UE individually. Firstly, it will increase energy consumption due to the isolated training of separate DRL agents. Secondly, training each DRL agent from the beginning will be either time-consuming or impossible to achieve satisfactory results. Therefore, it is logical to consider efficiently training a DRL agent as the preferred option. While it is possible to achieve optimal performance by training the DRL agent individually for each UE or UAV, it is also viable to use distributed training. Moreover, many effective DRL techniques are impractical considering the privacy concerns in edge computing systems, networking limitations, and the difficulty of handling non-independent and identically distributed data. Thus, this work introduces federated learning for distributed training.

Manuscript received 27 July 2023; revised 3 February 2024, 31 March 2024, and 18 May 2024; accepted 20 May 2024. Date of publication 22 May 2024; date of current version 17 October 2024. This work was supported by the Outstanding Potential for Excellence in Research and Academics (OPERA) Award, Birla Institute of Technology and Science Pilani under Grant FR/SCM/27-May-2020/EEE. The review of this article was coordinated by Prof. Rongqing Zhang. (Corresponding author: Sandeep Joshi.)

The authors are with the Department of Electrical and Electronics Engineering, Birla Institute of Technology and Science Pilani, Pilani 333031, India (e-mail: agarwalnipun2@gmail.com; sandeep85joshi@gmail.com).

Digital Object Identifier 10.1109/TVT.2024.3404223

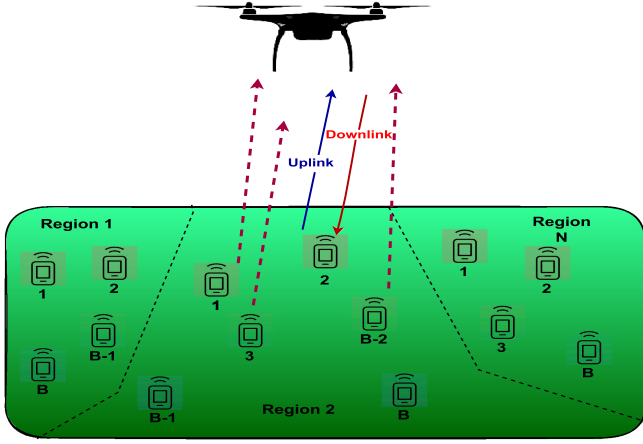


Fig. 1. System architecture with a UAV over a field and several UEs positioned in subdivided zones. The UAV receives connection requests from various UEs, which are represented by the dashed arrow lines, and the solid arrow lines represent the uplink and downlink transmission links.

We consider a MEC system consisting of a UAV with a computing-caching-communication device and several UEs stationed on the ground to accommodate dynamic and time-varying communication scenarios. Our major contributions are summarized as follows.

- We formulate an optimization problem for the time-slotted UAV-assisted MEC system, which incorporates designs for user scheduling, UAV mobility, resource allocation, and transmission delay. To reduce the processing time, a non-convex computation offloading problem is treated as a Markov decision process (MDP). In particular, we consider the energy constraints and the latency limitations imposed on the computation by the embedded processors in the UAV.
- We propose a decentralized federated averaging (DFedAvg) approach for offloading computations and determining the optimal policy for user scheduling, UAV mobility, and resource allocation in a UAV-assisted MEC network.
- With extensive simulations, considering different system parameter settings, we show that the proposed approach, compared to the existing baseline algorithms, provides the most reliable and best performance. Furthermore, the proposed method provides a high degree of flexibility and agility for implementing a collaborative machine learning-based approach within the UAV networks.

The remainder of the paper is organized as follows. Section II describes the system architecture of the considered UAV-assisted MEC network. Section III gives the optimization formulation and the proposed algorithm. The simulation results and comparisons with the baseline models are presented in Section IV. Finally, Section V concludes the paper.

II. SYSTEM ARCHITECTURE

We consider an MEC network aided by a UAV equipped with a nano MEC server consisting of B UE nodes. We assume that only one UE at a time can communicate with the UAV. A schematic diagram illustrating the considered network is shown in Fig. 1. As UE has a restricted amount of processing power, it delegates some of its computing responsibilities to the MEC server housed within the UAV. Every UE has time-shared access to the computational resources made available by the UAV [9]. It is assumed that the UAV will move throughout the territory at a slow speed and in a random, unpredictable manner. At the beginning of each time slot, the UAV will first maintain a stationary hovering posture and

then commence communication with one of the stationary UE. After initially delegating some of its computational tasks to a server, the UE will complete the remaining tasks on its local computing system. Energy is required for the various stages of operation, including flight, processing, and sending the signal back to the UE. The total delay accounts for the time required for up-linking between the UE and the UAV, as well as the computation time taken by the UAV. Following this, the computed data is transmitted back to the UE using the downlink transmission, as shown in Fig. 1. The downlink transmission has no role to play in the formulation of the optimization problem. The UAV hovers over the region at a fixed height, H , and randomly moves between regions for a fixed interval of time, and the total time of the flight is slotted into multiple time slots, $\tau \in (1, 2, 3, \dots, T_s)$, where T_s denotes the total number of time slots. The coordinates of the UAV are specified by $\rho(\tau) = [x(\tau), y(\tau)]^T \in \mathbf{R}^{2 \times 1}$, where $[\cdot]^T$ denotes the transpose. The UE coordinates are specified by $\kappa_\beta(\tau) = [x_\beta(\tau), y_\beta(\tau)]^T$, where $\beta \in (1, 2, 3, \dots, B)$, with B denoting the total number of UEs. The channel gain between the UAV and the UE considering line-of-sight communication [7] is given by

$$z_\beta(\tau) = \frac{\xi_o}{\|\rho(\tau) - \kappa_\beta(\tau)\|^2 + H^2}, \quad (1)$$

where ξ_o is the channel gain at a reference distance of 1 m and $\|\cdot\|^2$ denotes the Euclidean distance. The UE off-load a portion of data to be processed to the UAV, and after processing, the data is sent back to the UE, where it is merged with the portion of data that has been processed locally by the UE. The data rate for β th UE during τ th time slot is expressed as

$$\phi_\beta(\tau) = F \log_2 \left(1 + \frac{P_{up} z_\beta(\tau)}{\sigma^2 + c_\beta(\tau) P_{loss}} \right), \quad (2)$$

where F denotes the bandwidth in Hz, the uplink power is denoted by P_{up} , P_{loss} denotes the uplink path loss, σ^2 represents the noise power, and $c_\beta(\tau)$ is the indicator of whether there is a blockage between the UAV and β th UE at time slot τ , i.e., 0 representing no blockage and 1 representing blockage [4]. In the considered MEC system, the tasks performed by the UE during each time slot are handled using a partial offloading method [10]. We assume that the computation tasks are perfectly separable and UAV establishes the communication link with a single UE in a geographical area. The local time execution delay for the β th UE at any time slot τ is given as

$$t_{local} = \frac{(1 - R_\beta(\tau)) \times P_\beta(\tau) \times \text{cyc}}{f_{user}}, \quad (3)$$

where $R_\beta(\tau) \in [0, 1]$ indicates the ratio of work that has been transferred to the server, $P_\beta(\tau)$ is the total computing task size for the β th UE, cyc is the processing unit cycle, and f_{user} is the computing power. For example, if there are a total of 10 UEs in a particular area, and the UAV connects to the 4th UE, then the value of β is taken as 4. The UAV is constantly in motion, and the location of the UAV at τ th time slot is expressed as

$$\rho(\tau) = \theta^Q [x(\tau), y(\tau)]^T + \theta^\mu V(\tau) + \theta^\alpha g(\tau), \quad (4)$$

where $[x(\tau), y(\tau)]$ denotes the initial position of UAV, $V(\tau)$ denotes the initial velocity of the UAV, $g(\tau)$ is the initial angle of elevation of the UAV, and θ^Q , θ^μ , and θ^α are learning hyper-parameters required in the model's learning process as explained in the next section. The amount of energy used by the UAV while in flight is expressed as [5]

$$A_{fly, \beta}(\tau) = \Gamma \|V(\tau)\|^2, \quad (5)$$

where $\Gamma = \text{mass of UAV} \times \text{time for flight}$. As the primary role of the UE is to upload a proportion of the data (generally $< 50\%$) to the UAV server for processing, which does not alter the optimization problem formulation, therefore, the proposed model does not account for the energy consumption of the UEs. The UE then implements the DFedAvg algorithm locally on the rest of the data. Hence the power consumption of the UE only depends on the task size left with the UE after off-loading to the UAV. The time taken by the MEC server to process requests can be broken down into two components, one being the delay in transmission, which can be expressed as

$$\Lambda = \frac{R_\beta(\tau) \times P_\beta(\tau)}{\phi_\beta(\tau)}. \quad (6)$$

The other component is the lag that occurs as a consequence of the computation that takes place on the MEC server and can be expressed as

$$\Xi = \frac{R_\beta(\tau) \times P_\beta(\tau) \times \text{cyc}}{f_{mec}}, \quad (7)$$

where f_{mec} denotes the computation frequency of the MEC server on the UAV. Similarly, the amount of energy that must be expended to offload the computing tasks to the server during the allotted time slot can also be split into two parts: the first is used for transmission, and the second is used for computations. When the analysis is carried out on the server located at MEC, similar to [11], [12], the amount of power that is consumed is given as $P_{OAV} = B \times f_{mec}^2$, and the energy consumption of UAV is expressed as

$$E_{OAV} = f_{mec}^2 \times R_\beta(\tau) \times P_\beta(\tau) \times \text{cyc}. \quad (8)$$

III. PERFORMANCE ANALYSIS

In this section, we describe the optimization problem, modelled as MDP, and the proposed algorithm, along with the steps taken to construct the federated learning technique. To solve the MDP, the state is represented by the model parameters of each local client in a given round. To update the global model, the federated learning model takes the current state into account and learns a parameter distribution following the optimization problem while calculating the aggregated weights belonging to each client. Then, the local clients, in this instance, the UE, get the updated global model parameters. The parameter updation following the DFedAvg algorithm, with the learning rate λ with a value equal to 0.01, at time instance $(t + 1)$ is described as

$$\begin{aligned} \theta^{Q'}(t+1) &= \theta^Q(t) + \lambda \Delta\theta^{Q'}(t) \\ \theta^{\mu'}(t+1) &= \theta^\mu(t) + \lambda \Delta\theta^{\mu'}(t) \\ \theta^{\alpha'}(t+1) &= \theta^\alpha(t) + \lambda \Delta\theta^{\alpha'}(t). \end{aligned} \quad (9)$$

The parameter $(\theta^Q, \theta^\mu, \theta^\alpha)$ values are initialized randomly and are not necessarily zero. The value for variables $\Delta\theta^{Q'}(t)$, $\Delta\theta^{\mu'}(t)$, and $\Delta\theta^{\alpha'}(t)$ are obtained from the difference of values of the variables $\theta^{Q'}$, $\theta^{\mu'}$, and $\theta^{\alpha'}$, respectively after every second. Initially, the value is randomized, and assuming that there are no past values, the values of $\Delta\theta^{Q'}(t)$, $\Delta\theta^{\mu'}(t)$, and $\Delta\theta^{\alpha'}(t)$ parameters are set to zero, but for later time instance the values update as a difference of the previous values of $\Delta\theta^{Q'}(t)$, $\Delta\theta^{\mu'}(t)$, and $\Delta\theta^{\alpha'}(t)$. This particular arrangement of considering the past values for the variables adds a layer of sequential implementation in the algorithm. We formulate the optimization problem to ensure that the limited computational resources are used efficiently; therefore, one of our goals is to reduce the maximum processing latency for all UEs. To achieve this, we simultaneously

Algorithm 1: Federated Learning Algorithm.

- 1: Set the iteration index, epoch = 0; and initialize a feasible solution; $\vartheta_\beta(\tau)$, $\rho(\tau)$, $R_\beta(\tau)$ for the optimization problem in (10)
- 2: **Repeat**
- 3: Set epoch = epoch + 1
- 4: **Update** $\rho(\tau)$ according to (4)
- 5: **Update** $\theta^Q, \theta^\mu, \theta^\alpha$ according to (9)
- 6: **Update** $\phi_\beta(\tau)$ according to (2)
- 7: **Update** $R_\beta(\tau)$ as

$$A_{fly,\beta}(\tau) + E_{OAV} = \log_2(B_{OAV})$$

- 8: **Update** τ_{delay} as

$$\sum_{\beta=1}^B \vartheta_\beta(\tau) \max[t_{local}, \Lambda + \Xi] = \log_2(B)$$

- 9: **Solve** the minimization problem in (10) to update $\vartheta_\beta(\tau)$, $\rho(\tau)$, $R_\beta(\tau)$

- 10: **Until** $\frac{\tau_{delay}(\text{epoch}) - \tau_{delay}(\text{epoch} - 1)}{\tau_{delay}(\text{epoch} - 1)} \leq \epsilon$,

where ϵ is a small positive constant.

optimize user scheduling, UAV mobility, and resource allocation within the system. The optimization problem is stated as

$$\begin{aligned} \min_{\vartheta_\beta(\tau), \rho(\tau), R_\beta(\tau)} & \sum_{\tau=1}^{T_s} \sum_{\beta=1}^B \vartheta_\beta(\tau) \max[t_{local}, \Lambda + \Xi] \\ \text{s.t. } & \vartheta_\beta(\tau) \in [0, 1], \forall \tau \in [1, T_s], \forall \beta \in [1, B], \\ & \sum_{\beta=1}^B \vartheta_\beta(\tau) = 1, 0 < R_\beta(\tau) < 1, \forall \tau, \beta, c_\beta(\tau) \in [0, 1], \forall \tau, \beta, \\ & \sum_{\tau=1}^T (A_{fly,\beta}(\tau) + E_{OAV}) \leq B_{OAV}, \forall \beta, \end{aligned} \quad (10)$$

where $\Lambda + \Xi$ is the total delay induced by the UAV, B_{OAV} is the total energy capacity of the UAV [11], and $\vartheta_\beta(\tau)$ denotes the total normalized task for each UE. The model incorporates federated learning as a solution to the problem of resource allocation and offloading decisions.

Due to its distributed nature, federated learning has the potential to enhance training performance while simultaneously protecting learners' privacy and safety. We use federated learning to optimize the model so that we can reduce the personalized characteristics of each local device model, improve the universality of the model, and be able to directly apply the model to new users, thereby reducing the amount of time needed for new users to go through the process of model training. The proposed solution using a federated learning algorithm is described by Algorithm 1. Other constraints include that the UAV must be flying inside a specific zone.

IV. NUMERICAL RESULTS

In this section, through numerical simulations, we analyze the proposed federated learning-based algorithm for computation offloading in the UAV-assisted MEC system. We used Keras API, Tensorflow 2.1.0, and Python 3.7 during the development process of the DFedAvg models.

TABLE I
SIMULATION PARAMETERS

Parameters	Value	Parameters	Value
B	10	L, W, H	100 m (each)
UAV weight	9.65 kg	$v_{\max}$	50 m/s
P_{loss}	20 dB	ξ_o	-50 dB
F	1 MHz	σ^2	-100 dBm
B_{UAV}	500 kJ	cyc	1000 cycles/bit
f_{UAV}	0.6 GHz	f_{mec}	1.2 GHz

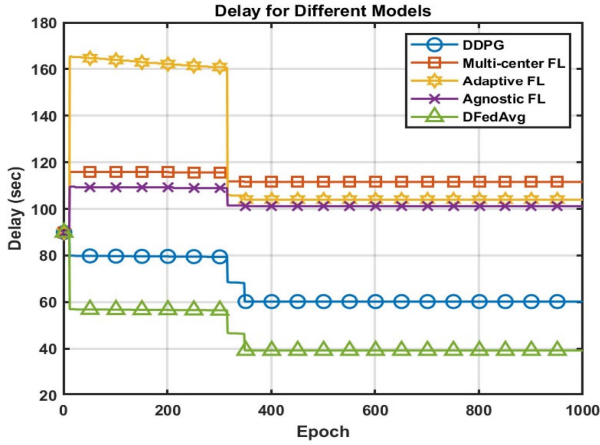


Fig. 2. Average delay versus the number of epochs for various machine learning models.

The performance of the proposed federated learning-based framework is compared with the performance of various baseline models for benchmarking. All models are subjected to exactly identical testing circumstances, and this includes the air-to-ground channel. The total experiment time is considered to be 400 seconds, with 40 equal time slots. The UE is limited in power, hence the uplink power P_{up} is kept at a minimal level of 0.1 W [13], [14]. For the MEC chip placed on the UAV, the cycles per bit, $\text{cyc} = 1000$ cycles/bit [5], [6]. The detailed simulation parameters are listed in Table I. Multiple ML models based on reinforcement learning and federated learning [8], [15], [16], [17] have been simulated to benchmark the proposed technique. We trained the proposed federated learning algorithm for one thousand iterations.

Fig. 2 shows the plot of average delay versus the number of epochs for different models. We can deduce from the plots that despite all of the algorithms reaching convergence, there are two points in this figure that are particularly noteworthy, i.e., the convergence point and the sudden shift in the delay value. Compared to the DDPG algorithm [8], the proposed algorithm achieves a lower delay and is constant for a longer time frame. We compare these methods using the delay results acquired after the algorithm converged. The corresponding results are shown in Fig. 3, where the delay of the DFedAvg algorithm is consistently the shortest amongst the other algorithms when considering the same task size. The overall delay, which includes the uplink and the processing time, grows as the size of the tasks performed on the UE increases. This is because more tasks are off-loaded to the UAVs. Considering that the DDPG algorithm [8] is the second best after our algorithm,

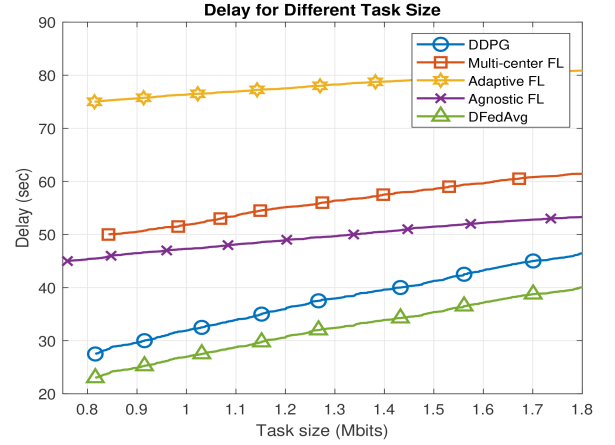


Fig. 3. Average delay versus the task size for various machine learning models.

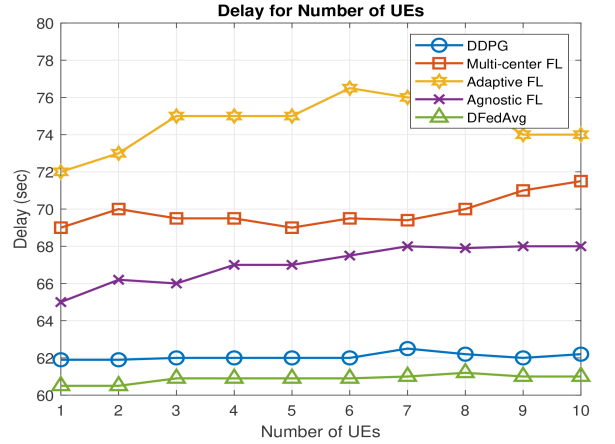


Fig. 4. Average delay for various machine learning models versus the number of UEs.

the delay difference demonstrates an improvement of 16.677%, and the performance of the proposed algorithm is superior as compared to the other models in the federated learning family. We evaluate the average processing latency between the proposed DFedAvg scheme, DDPG [8], Multi-center [15] federated learning model, Adaptive [16] federated learning model, and Agnostic [17] federated learning model in Fig. 4, where the number of UEs range from one to ten in a particular area. We assume that the overall size of the work that has to be finished in a certain amount of time is the same regardless of the number of UEs used. As can be seen from Fig. 4, the average processing delay is virtually the same for DFedAvg and DDPG models, while all the other models [15], [16], [17] exhibit an irregular delay pattern. Moreover, the DFedAvg technique has the shortest delay on average out of all. This is because the DFedAvg method can locate the best solution in the action space and acquire the best control policy.

Figs. 5 and 6 show how the system's computational latency is affected when tasks are off-loaded to other processors. The proposed model is compared with the DDPG model. Fig. 5 shows the advantage of off-loading data and the delay. It is clear that transferring more than half of the total data to the UAV significantly increases the delay experienced by the systems as a whole. We can see from Fig. 6 that as the task size increases, a lesser proportion of the work is transferred to the UAV. This is done to reduce the overall delay brought on by the uplink and the computation. As shown in Fig. 6, for a realistic application range

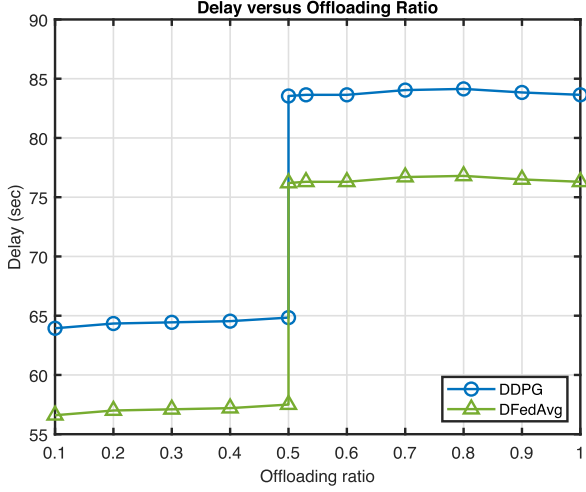


Fig. 5. Delay versus the offloading ratio for the DDPG and the proposed algorithm for a given task.

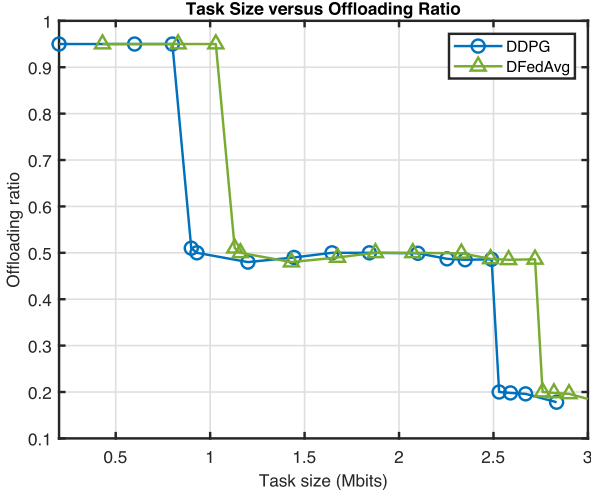


Fig. 6. Offloading ratio versus the task size for the DDPG and the proposed algorithm.

of data between 0.5 and 3 Megabits, about fifty percent of the data is off-loaded onto the MEC-aided UAV. We can also observe from Fig. 6 that the off-loading process happens faster when there is more work to be done. The critical justifications for this operation are centered on the channel capacity as well as the channel delay. As the amount of data increases, so does the need for a larger bandwidth in the communication channels. This, in combination with the system's inherent instability, i.e., a UAV constantly altering position, results in an accumulation of delays. Figs. 5 and 6 illustrate the similarities between the presented and the DDPG model.

Fig. 7 illustrates the comparison of total UAV power consumption between the proposed model, the DDPG model [8], and a couple of baseline models addressing the issue of computation off-loading in multi-UAV settings at various CPU frequencies, here one CPU frequency is completed in 1 ms. We give a comparison with FLR [18], which deals with the computation offloading management in a swarm of drones using a rating-based federated learning approach based on Multi-center federated learning [15] and PF-MADDPG [19], which introduces a personalized federated DRL-based trajectory optimization

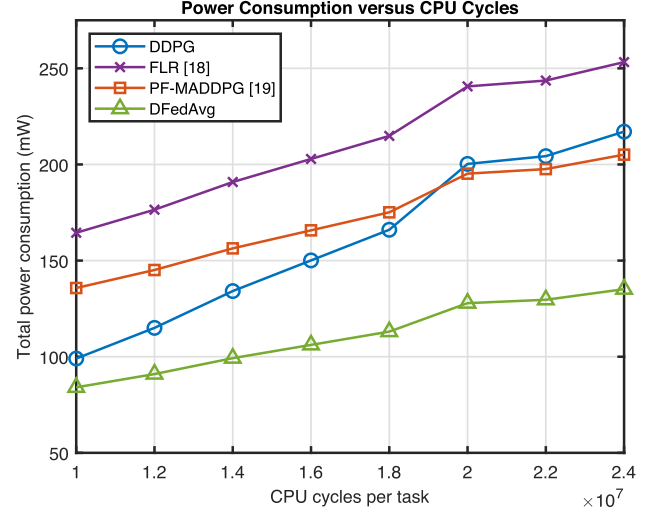


Fig. 7. Total UAV power consumption versus CPU cycles.

for a multi-UAV scenario which uses a modified version of Agnostic federated learning [17]. From the plot, it is evident that the proposed DFedAvg scheme achieves an overall better performance in terms of power consumption in comparison to all the other schemes. The proposed scheme is $\sim 25\%$ more energy efficient for lower CPU cycles and $\sim 35\%$ energy efficient for the highest possible CPU cycle. This is because the DFedAvg scheme can generate a range of continuous smaller subsets of actions, unlike the limited set of discrete actions in other schemes. Consequently, DFedAvg can accurately identify crucial factors, such as offloading ratio and task size, that significantly affect the energy consumption in the continuous action control system.

V. CONCLUSION

In this work, we presented the concept of computation offloading in a UAV-assisted MEC network where a flying UAV offers UEs communication and computing services. By conjointly optimizing the user scheduling, task offloading ratio, and flight speed, we proposed to reduce the total amount of time lost due to maximum processing delays throughout the time. We have provided the federated learning-based DFedAvg method to acquire the optimum offloading strategy to solve the non-convex optimization problem with discrete variables, which allowed us to find the best solution. We evaluated the functionality of the DFedAvg algorithm through extensive numerical simulations and compared the effect of various factors, such as off-loading ratio and number of UEs. We demonstrated that the proposed algorithm achieves better performance in processing latency when compared to standard baseline methods, both as a function of the size of the task and the number of UEs. Furthermore, the average delay for multiple UEs is also the least for the proposed algorithm, which attains a stable value. The proposed model is a primary model where we have assumed only one UE to be connected to one UAV. If there are multiple UAVs with the same configuration as we have considered, this model can be scaled to serve multiple UE at the same time. The proposed model can be extended to solve bandwidth restrictions by creating a new data intermediate layer to accommodate the transmission of greater data sizes with measures to safeguard the data while it is offloaded. Another possible extension is to consider a scenario with a broad set of multi-users and numerous edges of the MEC systems, considering users' mobility between edge servers.

REFERENCES

- [1] Q.-V. Pham et al., "A survey of multi-access edge computing in 5G and beyond: Fundamentals, technology integration, and state-of-the-art," *IEEE Access*, vol. 8, pp. 116974–117017, 2020.
- [2] J. Zhang et al., "Joint resource allocation for latency-sensitive services over mobile edge computing networks with caching," *IEEE Internet Things*, vol. 6, no. 3, pp. 4283–4294, Jun. 2019.
- [3] T. X. Tran and D. Pompili, "Joint task offloading and resource allocation for multi-server mobile-edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 1, pp. 856–868, Jan. 2019.
- [4] L. Ge, P. Dong, H. Zhang, J.-B. Wang, and X. You, "Joint beamforming and trajectory optimization for intelligent reflecting surfaces-assisted UAV communications," *IEEE Access*, vol. 8, pp. 78702–78712, 2020.
- [5] S. Zhang, L. Zhang, F. Xu, S. Cheng, W. Su, and S. Wang, "Dynamic deployment method based on double deep Q-network in UAV-assisted MEC systems," *J. Cloud Comput.*, vol. 12, no. 1, 2023, Art. no. 130.
- [6] F. Xu, S. Wang, W. Su, and L. Zhang, "Joint optimization task offloading and trajectory control for unmanned-aerial-vehicle-assisted mobile edge computing," *Comput. Elect. Eng.*, vol. 111, Oct. 2023, Art. no. 108916.
- [7] Q. Tang, C. Dai, Z. Yu, D. Cao, and J. Wang, "An UAV and EV based mobile edge computing system for total delay minimization," *Comput. Commun.*, vol. 212, pp. 104–115, Dec. 2023.
- [8] C. Qiu, Y. Hu, Y. Chen, and B. Zeng, "Deep deterministic policy gradient (DDPG)-Based energy harvesting wireless communications," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8577–8588, Oct. 2019.
- [9] Y. Dai, D. Xu, S. Maharjan, G. Qiao, and Y. Zhang, "Artificial intelligence empowered edge computing and caching for internet of vehicles," *IEEE Wirel. Commun.*, vol. 26, no. 3, pp. 12–18, Jun. 2019.
- [10] U. Saleem, Y. Liu, S. Jangsher, and Y. Li, "Performance guaranteed partial offloading for mobile edge computing," in *Proc. IEEE Glob. Commun. Conf.*, 2018, pp. 1–6.
- [11] Y. Wang, M. Sheng, X. Wang, L. Wang, and J. Li, "Mobile-edge computing: Partial computation offloading using dynamic voltage scaling," *IEEE Trans. Commun.*, vol. 64, no. 10, pp. 4268–4282, Oct. 2016.
- [12] D. P. Mtowe and D. M. Kim, "Edge-Computing-Enabled Low-Latency communication for a wireless networked control system," *Electronics*, vol. 12, no. 14, Jul. 2023, Art. no. 3181.
- [13] J. Nie and S. Haykin, "A Q-learning-based dynamic channel assignment technique for mobile communication systems," *IEEE Trans. Veh. Technol.*, vol. 48, no. 5, pp. 1676–1687, Sep. 1999.
- [14] M. Coldrey, J.-E. Berg, L. Manholm, C. Larsson, and J. Hansryd, "Non-line-of-sight small cell backhauling using microwave technology," *IEEE Commun. Mag.*, vol. 51, no. 9, pp. 78–84, Sep. 2013.
- [15] G. Long, M. Xie, T. Shen, T. Zhou, X. Wang, and J. Jiang, "Multi-center federated learning: Clients clustering for better personalization," *World Wide Web*, vol. 26, no. 1, pp. 481–500, Jan. 2023.
- [16] Y. Deng, M. M. Kamani, and M. Mahdavi, "Adaptive personalized federated learning," 2020, *arXiv:2003.13461*.
- [17] M. Mohri, G. Sivek, and A. T. Suresh, "Agnostic federated learning," in *Proc. 36th Int. Conf. Mach. Learn.*, 2019, pp. 4615–4625.
- [18] D. Rahbari, M. M. Alam, Y. L. Moullec, and M. Jenihhin, "Fast and fair computation offloading management in a swarm of drones using a rating-based federated learning approach," *IEEE Access*, vol. 9, pp. 113832–113849, 2021.
- [19] Z. Song, C. Ma, M. Ding, H. H. Yang, Y. Qian, and X. Zhou, "Personalized federated deep reinforcement learning-based trajectory optimization for multi-UAV assisted edge computing," in *Proc. IEEE/CIC Int. Conf. Commun. China*, 2023, pp. 1–6.